PROJETO FINAL — POO II - Sistema de Pedidos e Pagamentos Extensível

Objetivo do projeto

Construir um sistema que cria pedidos, calcula total, aplica descontos, escolhe forma de pagamento e registra logs — com arquitetura extensível (sem `if/switch` espalhado) e usando DI como no mercado.

Requisitos de entrega

Você deve entregar:

1. Projeto compilando e rodando
2. Código organizado em pastas (Domain/Application/Infrastructure)
3. Um `Program.cs` com um cenário demonstrando o fluxo completo
4. README com instruções de execução e exemplos de saída

1) Domínio (Domain)

1.1 Entidades

`Produto`

- Propriedades:
  - `Id` (int)
  - `Nome` (string)
  - `Preco` (decimal)
- Regras:
  - `Nome` obrigatório
  - `Preco > 0`

`ItemPedido`

- Propriedades:
  - `Produto`
  - `Quantidade`
- Regras:
  - `Quantidade >= 1`
- Método:
  - `Subtotal()` = `Produto.Preco * Quantidade`

`Pedido`

- Deve conter itens internamente (lista não pode ser exposta mutável)
- Métodos:
  - `AdicionarItem(Produto produto, int quantidade)`
  - `TotalBruto()` (soma dos subtotais)

2) Regras de negócio como abstrações (Interfaces)

2.1 Descontos

Crie a interface:

```
public interface IDesconto
{
    string Nome { get; }
    decimal CalcularDesconto(Pedido pedido);
}
```

Implemente **no mínimo 2 descontos**:

1. **DescontoPercentual**
    - Recebe % no construtor
    - Ex.: 10% do TotalBruto
2. **DescontoPorQuantidade**
    - Se o pedido tiver **≥ 5 itens no total**, dá desconto fixo (ex.: R$ 20)

Importante: o sistema deve suportar novos descontos sem alterar o processador (OCP).

2.2 Pagamento

Crie a interface:

```
public interface IPagamento
{
    string Nome { get; }
    void Pagar(decimal valor);
}
```

Implemente **no mínimo 2 formas**:

- `PagamentoPix`
- `PagamentoCartao`

3) Camada de Aplicação (Application)

3.1 Processador de Pedido (classe abstrata + template)

Você deve ter um fluxo padrão de processamento (template method) para:

- Calcular total bruto
- Calcular total de descontos (somando todos os descontos)
- Calcular total final
- Efetuar pagamento
- Logar o resultado

Crie uma classe abstrata:

```
public abstract class PedidoProcessorBase
{
    public void Processar(Pedido pedido);
}
```

A ideia é: `Processar()` define o **fluxo padrão** e chama métodos protegidos/virtuais que podem ser especializados.

Exemplo de responsabilidades esperadas no fluxo:

- `ValidarPedido(pedido)`
- `CalcularTotalBruto(pedido)`
- `CalcularDescontos(pedido)`
- `CalcularTotalFinal(bruto, descontos)`
- `RealizarPagamento(valorFinal)`
- `RegistrarLog(...)`

Você deve criar uma implementação concreta:

- `PedidoProcessor`

Aqui você usa **classe abstrata** de forma realista: reaproveitar fluxo e padronizar comportamento.

3.2 Injeção de dependência (DIP)

O `PedidoProcessor` **não pode** instanciar descontos/pagamentos diretamente.

Ele deve receber no construtor:

- `IEnumerable<IDesconto>` (lista de descontos configurada)
- `IPagamento` (forma escolhida)
- `ILogger` (logger simples)

4) Infraestrutura (Infrastructure)

4.1 Logger

Crie:

```
public interface ILogger
{
    void Info(string message);
    void Error(string message);
}
```

Implemente `ConsoleLogger`.

4.2 Repositório Genérico (Generics)

Crie uma interface:

```
public interface IRepository<T>
{
    void Add(T entity);
    IEnumerable<T> GetAll();
}
```

Implemente:

- `InMemoryRepository<T>`

O repositório deve ser **genérico e reutilizável**.

(Opcional, recomendado) Constraint

Crie uma interface `IEntity` com `Id` e aplique:

```
where T : IEntity
```

5) DI / IoC

Use o container do .NET:

- `Microsoft.Extensions.DependencyInjection`

Você deve registrar:

- Logger
- Repositório
- Descontos
- Pagamentos
- Processor

E resolver o processor via container.

O `Program.cs` deve ficar “magro”: só montar cenário e rodar.

6) Regras obrigatórias

- Não pode usar `if/switch` no processor para escolher qual desconto aplicar
- Não pode instanciar implementação concreta dentro do processor (`new PagamentoPix()`)
- Deve aplicar polimorfismo via interfaces
- Deve aplicar SRP: classes pequenas e com responsabilidade clara
- Deve aplicar DIP: depender de abstrações, não de concretos

7) Cenário obrigatório

No `Program.cs`, crie:

- 3 produtos:
  - Mouse (79,90)
  - Teclado (159,90)
  - Headset (249,90)
- Um pedido com:
  - Mouse x2
  - Teclado x1
  - Headset x2 (total de itens = 5 → ativa desconto por quantidade)

Registre descontos:

- 10% percentual
- desconto por quantidade (R$ 20)

Escolha pagamento:

- Pix (ou Cartão, mas tem que ser configurável via DI)

Saída esperada (modelo)

```
Total bruto: 818,50
Descontos aplicados:
 - DescontoPercentual (10%): 81,85
 - DescontoPorQuantidade: 20,00
Total descontos: 101,85
Total final: 716,65
Pagamento: Pix
Status: Pago com sucesso
```

8) Bônus opcional

1. **Escolha de pagamento por argumento** (`args`) sem `if` no processor
   - Ex.: `--pagamento=pix` / `--pagamento=cartao`
   - Permitido `if` apenas no *composition root* (Program.cs), nunca no domínio.
2. **Reflection opcional**: auto-registrar todas classes que implementam `IDesconto` no container.